

เอกสารอธิบายระบบ Asset Management (Backend + Frontend)

ภาพรวมสถาปัตยกรรม

Stack ที่ใช้: Node.js + Express 5.2.1 (เซิร์ฟเวอร์), EJS (Server-Side Rendering), MySQL (ผ่าน mysql2), session เก็บใน DB (express-mysql-session), bcrypt (แฮชรหัสผ่าน), multer (อัปโหลดไฟล์), xlsx (อ่าน Excel), nodemailer (ส่งอีเมลผ่าน SMTP), helmet (security headers + CSP), custom rate-limit + custom CSRF

โครงสร้างโฟลเดอร์

```
app.js                <- ทางเข้าหลักของเซิร์ฟเวอร์

config/db.js          <- สร้าง/เชื่อมต่อ MySQL + กำหนด schema

middleware/           <- auth.js, csrf.js, locale.js, upload.js, rateLimit

routes/               <- auth, dashboard, upload, update, transfer, documents, logs,
profile, admin

controllers/          <- ตัวจัดการ logic ของแต่ละ route

models/               <- คำสั่ง SQL ของแต่ละตาราง

helpers/excelParser.js <- แปลง Excel -> rows

views/                <- ไฟล์ EJS (หน้าจอ + partials)

public/               <- css/js ของฝั่ง client

locales/              <- th.json, en.json (i18n)
```

PART 1 — BACKEND

1. จุดเริ่มต้น: app.js

เมื่อเซิร์ฟเวอร์ start ขึ้นมา จะทำงานตามลำดับ middleware ต่อไปนี้ (เรียงต่อกันเป็น chain):

- helmet — ตั้ง security headers รวมถึง CSP (Content-Security-Policy) ที่อนุญาตเฉพาะ CDN ของ Bootstrap / bootstrap-icons / SweetAlert2 จึงต้องโหลดผ่าน jsdelivr ไม่ใช่ local
- express.json() / express.urlencoded() — parse body ของ request (JSON + form)
- express-session + express-mysql-session — session ของผู้ใช้เก็บอยู่ในตาราง MySQL (ไม่ใช่หน่วยความจำ)
- Request logger — log ทุก request (method, url, status, duration, IP)
- res.locals helper — inject flash messages (error/success), csrfToken, RECAPTCHA_SITE_KEY ไปให้ทุก view
- Static files — เปิด service โฟลเดอร์ public/ (js, css)
- Route mounting — แบ่งตาม path: /auth/*, / (dashboard), /upload, /update, /transfer, /documents, /logs, /profile, /admin (ทุกชุดบังคับ middleware ตรวจ login/role)
- Error handler สุดท้าย — 404 + error 500 หน้าเว็บ

2. config/db.js — เชื่อมต่อและสร้างฐานข้อมูล

- สร้าง connection pool (ตั้งค่าใน .env)
- ฟังก์ชัน createDatabase() — รันตอน start ครั้งแรก
- สร้าง database asset_management ถ้ายังไม่มี
- สร้างตารางหลัก: users (username, email, password_hash, role, department, full_name, verified), assets (ตารางใหญ่ ~20 คอลัมน์ เช่น BUSINESS_UNIT, ASSET_ID, TAG_NUMBER, SERIAL_NUMBER_ASSET, DESCR, MODEL, PLANT, DEPTID, DEPT_NAME, CATEGORY, ASSET_STATUS, X_AGREEMENT_ID, EXPIRE_DATE, owner, updated_by, qr_code)
- departments (costcenter, name) และ categories (code, name)
- ตารางสนับสนุน: activity_logs (module, action, target, details), transfer_history (asset_id, from_dept, to_dept, transferred_by, note), documents (ไฟล์ PDF + department + uploader)

- migrations — ตรวจสอบว่าคอลัมน์ที่เพิ่มใหม่ (เช่น EXPIRE_DATE) มีอยู่หรือไม่ ถ้าไม่มีก็ ALTER TABLE เพิ่มให้
- ผู้ใช้ DB หลัก: asset@localhost เชื่อม database asset_management

3. Middleware (ทีละตัว)

auth.js

- requireAuth — ยังไม่ login redirect ไป /auth/login
- requireAdmin — role ต้องเป็น admin หรือ super_admin
- requireSuperAdmin — role ต้องเป็น super_admin เท่านั้น (จัดการผู้ใช้/แผนก)

csrf.js

- สร้าง token แบบ random (crypto) เก็บใน session ตอนโหลดหน้า
- ทุก POST/PUT/DELETE ต้องส่ง _csrf มาตรงกัน ถ้าไม่ตรง -> 403 (กัน CSRF attack)

locale.js

- อ่านภาษาจาก cookie (lang) หรือ header Accept-Language
- ตั้งค่า res.locals.__ และ req.lang -> view ใช้ __('key') ในการแปล

upload.js

- multer แบบ Excel: รับ .xlsx/.xls/.csv ขนาดสูงสุดกำหนดไว้ เก็บใน memory
- multer แบบ PDF: รับ .pdf
- multer แบบรูปโปรไฟล์: ตรวจสอบ mime + resize/เก็บไฟล์

อื่น ๆ

- ตัวจำกัด rate — login/register ถ้าพิมพ์ผิดหลายครั้งจะโดนหน่วงเวลา (กัน brute force)

4. Models (ทุกอันเป็นฟังก์ชันที่ดึงข้อมูลจาก DB)

- user.js — findByUsername, create, verify, findByEmail, updateProfile, updatePassword, countUsers, listUsers (รองรับ sort/page), delete
- asset.js — ฟังก์ชันหลัก:

- `getAll(filters, {limit, offset})` — SELECT พร้อม WHERE แบบ build-from-filter: search (LIKE หลายคอลัมน์: ASSET_ID, TAG_NUMBER, DESCR, SERIAL), status, dept, category, plant, owner แล้ว ORDER BY + LIMIT/OFFSET
- `countAggregate(filters)` — นับจำนวนรวมตาม filter เดียวกัน (สำหรับคำนวณ pagination)
- `getSummary()` — นับจำนวนสินทรัพย์ตามสถานะ (Active, Inactive, Scrapped) สำหรับการดบน dashboard
- `getById(id), getActivities(id)` — ประวัติการแก้ไขของเครื่องนั้น
- `updateDept` (เปลี่ยนแผนก/owner), `updateStatus`, `searchAssets` (สำหรับหน้าถ่ายโอน)
- `activityLog.js` — `log(module, action, target, details, items)` บันทึกทีละ batch, `getLogs(filters, {limit, offset})`, `countLogs`, `getBatchDetail(id)`
- `transfer.js` — `searchAssets(q)`, `createBulk(assetIds, toDept, note, user)` (UPDATE หลายแถว + เขียน `transfer_history` + `activityLog`), `getHistory(search, {limit, offset})`
- `document.js` — `insert` (ไฟล์+ชื่อจริง+ขนาด+dept+uploader), `list` (join ชื่อคนอัปโหลด + รองรับ filter dept/search/sort), `getById`, `deleteDoc`
- `department.js` / `category.js` — `create`, `list`, `findByName/code`, `upsert` (สำหรับ import Excel), `delete`

5. Routes + Controllers — ไล่ตามโมดูล

โมดูล	Routes	Controller / Model
Auth	/auth/*	authController / user
โปรไฟล์	/profile	profileController / user
Dashboard	/, /asset/:id, /export, /labels	dashboardController / asset
Upload	/upload/*	uploadController / asset
Update	/update/*	updateController / asset
Transfer	/transfer/*	transferController / transfer

Documents	/documents/*	documentController / document
Logs	/logs/*	logController / activityLog
Admin	/admin/*	adminController / user, department, category

5.1 Auth (/auth)

- register — ตรวจ username/email ไม่ซ้ำ, password ต้องซับซ้อน (ขั้นต่ำ 6 ตัว ต้องมีอักษร/ตัวเลข/ตัวพิมพ์ใหญ่), เลือกแผนก, ผ่าน reCAPTCHA v3 แล้ว -> เข้ารหัส bcrypt -> สร้างผู้ใช้แบบรอ verification token -> ส่งอีเมลยืนยันผ่าน SMTP (ลิงก์มี token)
- verify?token=... — ตรวจ token -> ตั้ง verified=1 -> login ได้
- login — ตรวจ username+password (bcrypt compare), เช็ค verified, rate-limit -> เก็บ session
- logout — ทำลาย session
- forgot / reset — ใส่ email -> ส่งลิงก์ reset (token หมดอายุ) -> ตั้งรหัสใหม่ (บังคับ complexity อีกครั้ง)
- profile — ดู/แก้ไขข้อมูลตัวเอง, เปลี่ยนรหัสผ่าน (ต้องรู้รหัสเก่า), อัปโหลดรูปโปรไฟล์

5.2 Dashboard (/) — หน้ากลาง

1. GET / -> รับ query params: page, limit (per-page), search, status, dept (จาก sidebar), category, plant, owner, sort
2. เรียก asset.getAll() + countAggregate() + getSummary() + ลิสต์สำหรับ dropdown (แผนก/หมวด/โรงงาน/ผู้ดูแล)
3. Render dashboard.ejs พร้อม pagination (คำนวณ totalPages)
 - GET /export — ดึงข้อมูลตาม filter ปัจจุบัน -> generate CSV ส่งดาวน์โหลด
 - GET /asset/:id — หน้า detail ครอบหุ้มคอลัมน์ + ประวัติการแก้ไข + ปุ่มแก้ไข/QR
 - GET /asset/:id/edit + POST — แก้ไขข้อมูลเครื่องนั้น ทุกครั้งบันทึก activity log
 - POST /labels/print หรือคล้าย — รับรายการ asset_id -> สร้างข้อมูลโหลดหน้า print label (tab ใหม่)

- POST /asset/:id/status — เปลี่ยนสถานะ (เช่น Active <-> Inactive) บันทึก log

5.3 Upload (/upload) — นำเข้า Excel ครั้งแรก (บุลก์)

1. GET /upload -> หน้าแบบฟอร์ม (ลากวางไฟล์) + ไซส์คอลัมน์ที่ระบบรองรับ
2. POST /upload -> multer รับไฟล์ .xlsx/.xls/.csv -> excelParser.parseAssetExcel(): จับคู่คอลัมน์ alias (เช่น ASSET_ID = Asset ID = รหัสสินทรัพย์) ผ่าน ALL_COLUMNS, อ่านทีละ sheet, ข้ามแถวว่าง
3. อ่านเสร็จ -> เก็บ preview ไว้ใน req.session -> render หน้า preview (ลิสต์สินทรัพย์ที่อ่านได้ + แถวแถวที่อ่านไม่ได้ + จำนวนที่จะ insert/update)
4. POST /upload/apply -> เอา preview จาก session -> INSERT แบบ upsert ลงตาราง assets + เพิ่ม department/category ที่ยังไม่มี + เขียน activity log แบบ batch -> flash success -> redirect

5.4 Update (/update) — อัปเดตจาก Excel พร้อม diff

1. GET /update -> หน้าเลือกไฟล์ (เหมือน upload)
2. POST /update/preview -> parse -> เทียบกับข้อมูลใน DB:
 - changed[] — asset_id มีอยู่แล้ว แต่ค่าบางคอลัมน์ต่าง -> สร้าง diff ที่ละหลายคอลัมน์ (แสดงค่าเก่า/ใหม่ ให้เลือกทีละคอลัมน์ว่าเอาของใหม่ (new) หรือเก็บของเก่า (old))
 - toCreate[] — asset_id ยังไม่มีใน DB -> เสนอเป็นรายการใหม่ (checkbox เลือกว่าจะเพิ่มใหม่)
 - unchanged[] — เหมือนเดิมทุกคอลัมน์
3. POST /update/apply -> อ่าน choice จาก form (choice_key_column=new|old, include_key=1) -> UPDATE/INSERT ตามที่เลือก -> log batch -> redirect พร้อมสรุปผล (updated/added/unchanged)

5.5 Transfer (/transfer) — ย้ายแผนกเป็นกลุ่ม

1. GET /transfer -> แบบฟอร์ม (ค้นหาสินทรัพย์ เลือกหลายรายการ) + ตารางประวัติการย้าย
2. GET /transfer/search?q= -> API เพื่อ autocomplete (LIKE), คืน JSON [{asset_id, descr, dept_name}]

3. POST /transfer/create -> รับ asset_ids[], to_dept (combobox), note -> UPDATE หลายแถว
เปลี่ยน DEPT_NAME (+ owner ถ้ามีการตั้ง) + เขียน transfer_history + activity log batch -> ป้องกัน
ไม่ให้เลือกไม่เลือกเลย/ไม่เลือกแผนก (SweetAlert ฝั่งหน้า)
4. ประวัติ: GET /transfer?search=&page=&limit= — filter + paginate

5.6 Documents (/documents) — เก็บ PDF แยกแผนก

- GET /documents — list ไฟล์, สลับ Grid/List view (บันทึกใน localStorage), ค้นหา (search), เรียง (sort_by/order), กรองแผนก (dept)
- GET /documents/upload -> multer PDF -> บันทึกไฟล์ (uuid ชื่อจริงเก็บใน DB) + ระบุ dept (super_admin เลือก dept ได้ ส่วน admin ปกติยึดแผนกตัวเอง)
- GET /documents/view/:id -> หน้า embed PDF (<embed> ใช้ path /documents/view/:id/file)
- GET /documents/download/:id -> res.download
- POST /documents/delete/:id -> ตรวจสอบสิทธิ์: super_admin ลบได้หมด, admin ลบได้เฉพาะ dept
ตนเอง

5.7 Logs (/logs) — ตรวจสอบประวัติการกระทำ

- GET /logs — ตาราง log: module (assets/transfer/documents/users), action (create/update/delete/transfer/clear), ค้นหา, paginate
- บันทึกเป็น batch: ถ้า action เกิดกับหลายรายการจะมีปุ่ม view items -> GET /logs/detail/:id แสดงรายการย่อย (asset_id, ชื่อ, สถานะตอนนั้น หรือเลขที่ถูกลบแล้ว)

5.8 Admin (/admin/*) — เฉพาะ super_admin

- Users tab: ตารางผู้ใช้ (sort ได้), ปุ่มแก้ไข (modal: username/email/full_name/role/department — ห้ามแก้ role ตัวเอง), เปลี่ยนรหัสผ่านให้คนอื่น, ลบผู้ใช้ (ต้องใส่ password ยืนยัน และลบตัวเองไม่ได้)
- Departments tab: เพิ่มแผนก (costcenter + name), import Excel (อ่าน sheet แผนก -> upsert), ลบ
- Categories tab: เพิ่มหมวด (code + name), import Excel, ลบ

6. helpers/excelParser.js

- นิยาม ALL_COLUMNS (26 คอลัมน์ตามไฟล์ต้นทาง) + map alias ต่าง ๆ
- parseAssetExcel(buffer) -> {assets:[], skippedRows, skippedSheets} ตรวจ header ทุก sheet, ข้ามแถวว่าง, normalize ค่า (trim, null -> "")
- parseDepartmentExcel() / parseCategoryExcel() -> คล้ายกันสำหรับ import แผนก/หมวด

7. ความปลอดภัยโดยรวม (backend)

- bcrypt สำหรับแฮชรหัสผ่าน
- CSRF token ต่อ session
- helmet + CSP จำกัดแหล่ง script (CDN เท่านั้น)
- rate-limit หน้า login/register
- role-based access (user/admin/super_admin)
- EJS escape ค่าทุกจุด (<%= %> ไม่ใช่ <%- %>)
- session เก็บใน MySQL (server restart ไม่หลุด)
- captcha ตอนสมัคร + ยืนยันอีเมลก่อนเข้าใช้
- migration อัตโนมัติ

PART 2 — FRONTEND

1. โครงสร้าง Views (EJS)

ทุกหน้าหลัง login มีโครงเหมือนกันผ่าน **partials**:

header.ejs

- เปิด `<head>`: ตั้ง `lang` ตามภาษา, โหลด Bootstrap CSS / bootstrap-icons / style.css / SweetAlert2 ผ่าน CDN
- สคริปต์กัน FOUC แบบ dark mode: อ่าน `localStorage.am-theme` หรือ `prefers-color-scheme` แล้วใส่คลาส `dark-mode` ตั้งแต่ก่อน render เนื้อหา

navbar.ejs

- แถบบนสุด: ปุ่ม hamburger (มือถือ) / ปุ่มสลับ sidebar (จอใหญ่), brand, ปุ่ม toggle ภาษา (TH/EN), ปุ่ม theme (พระจันทร์/พระอาทิตย์), เมนูโปรไฟล์ + ออกจากระบบ

sidebar.ejs

- เมนูซ้าย แยกตาม role: สมาชิกทั่วไป (Dashboard, Upload, Update, Transfer, Documents), admin เพิ่ม Logs, super_admin เพิ่ม Admin
- dropdown กรองแผนก (super_admin เท่านั้น) พร้อมช่องค้นหาแบบ filter-as-you-type
- สถานะ collapsed/expand บันทึกใน localStorage

footer.ejs

- ปิด `<body>`: Bootstrap JS bundle, sidebar.js, dept-combo.js
- สคริปต์ global submit handler: ถ้าฟอร์มมี data-confirm -> ยกเลิก submit ธรรมดา -> เปิด SweetAlert ยืนยัน -> กดยืนยันแล้วค่อย submit (ใช้กับปุ่มลบ/ถ่ายโอนทุกที่)
- สคริปต์ toggle dark mode

2. หน้าหลัก dashboard.ejs

- การ์ดสรุป (getSummary): จำนวนสินทรัพย์ทั้งหมด / ใช้งาน / ไม่ใช้งาน / scrapped ฯลฯ

- Filter bar (#filterForm, method GET): ช่องค้นหา (search), ตัวกรองสถานะ/แผนก/หมวด/โรงงาน/ผู้ดูแล, ปุ่ม Apply/Reset
- Per-page (#limitSelect): select 10/25/50/100 — ผูกเป็น JS event listener (ไม่ใช่ onchange inline) -> เซ็ต hidden #filterPage=1 แล้ว form.submit() เพื่อกลับหน้าแรกเสมอ (commit 6b0f33a)
- ตารางสินทรัพย์อยู่ใน .table-scroll-wrapper (commit 6b0f33a):
- ช่องใช้ max-height: 70vh; overflow: auto -> เลื่อนแนวตั้งในกล่อง
- thead th เป็น position: sticky; top: 0; z-index: 2 -> หัวตารางติดค้างตลอดแม้เลื่อนลง
- ข้างในเก็บ min-width เพื่อให้ยังเลื่อนแนวนอนได้เหมือนเดิม (ตาราง ~26 คอลัมน์)
- แต่ละแถว: checkbox (เลือกหลายชิ้น), ASSET_ID (ลิงก์ไปหน้า detail), ชื่อ/รุ่น/แผนก/สถานะ (badge สี), ปุ่ม: รายละเอียด, แก้ไข (modal), QR code (modal), Print label
- Pagination: ตัวเลขหน้า + prev/next, หน้าปัจจุบัน active, ตัวเลขแบบมี ... เมื่อเยอะ
- Modals: โปรไฟล์ (ดู/แก้), แก้ไขสินทรัพย์, QR modal (แสดง QR จาก renderQR() บันทึก/ดาวน์โหลด/ปริ้นต์), ยืนยันลบ
- สคริปต์ inline: เช่น checkbox เลือกหมู่ ย้ายสถานะที่จะหลายชิ้น

3. ไฟล์ JavaScript ฝั่ง client

- sidebar.js — จัด sidebar: collapse/expand จอใหญ่, drawer + overlay จอมือถือ, sync กับ localStorage และ resize
- dept-combo.js — 2 ฟังก์ชัน:
 1. ช่องค้นหากรองแผนกใน sidebar dropdown (filter รายการ, Enter เลือกตัวแรก, focus หลังเปิด dropdown)
 2. combobox เลือกแผนกบนหน้า transfer (select ช้อน + text input + ลิสต์ผล, ArrowUp/Down เลื่อน, Enter เลือก, Esc ปิด)
- qr.js — สร้าง QR (ใช้ lib qrcode.js): renderQR(container, text), printLabel/printLabels เปิด tab ใหม่ window.open + document.write HTML ของสติ๊กเกอร์ขนาด 70x24mm (@page size: 70mm 24mm) แสดง tag/descr/agreement/vendor/expire + รูป QR

- หมายเหตุ bug ที่ค้างอยู่: qr.js:79 มี
`<script>window.onload=function(){window.print()}</script>` — `window.print()` เป็น
synchronous/blocking ทำให้หน้าต่างเดิม freeze จนกว่าจะปิด popup (ต้องแก้เป็นปุ่ม Print แบบ
manual)

4. CSS (public/css/style.css)

- ใช้ CSS variables แยก light/dark theme (--bg, --card, --text) และคลาส dark-mode บน `<html>`
สำหรับสลับค่า
- คลาส .table-scroll-wrapper (sticky thead + scroll แนวตั้ง), ปุ่ม sidebar, badges สถานะ, สไตลการ์ด
เอกสาร และอื่น ๆ
- สไตล print ของ label อยู่ใน qr.js ไม่ใช่ไฟล์นี้

5. ระบบภาษา (i18n)

- ไฟล์ locales/th.json + en.json; ฟังก์ชัน __('key') inject เข้าทุก render ผ่าน middleware locale.js
- สลับภาษาจาก navbar (เช็ค cookie lang) -> reload หน้าใหม่เป็นภาษาที่เลือก; ข้อความทั้งหมด (label, placeholder, alert, title/sweetalert) อ่านผ่าน __
- ตัวอย่าง: __('dashboard.filter.apply'), __('transfer.confirm', count, dept) (รองรับพารามิเตอร์
%s/%d)

ตัวอย่างการทำงานทั้งรอบ (request cycle)

1. ผู้ใช้ login -> session ถูกสร้างใน MySQL -> request ถัดไปส่ง cookie connect.sid
2. เปิด / -> requireAuth เช็ค session (ได้ user_id/role) -> dashboardController เรียก model -> SQL (filter + limit/offset + count) -> render dashboard.ejs ด้วย EJS (escape + __ แปลภาษา + ฝัง csrfToken)
3. ผู้ใช้กด Apply filter -> เกิด GET ใหม่พร้อม query -> วนกลับข้อ 2
4. ผู้ใช้แก้ไขเครื่อง -> modal submit POST /asset/:id/edit พร้อม _csrf -> csrf.js ตรวจ token -> controller UPDATE + เขียน activity log -> redirect พร้อม flash success
5. ผู้ใช้ select 3 รายการ + print label -> หน้า dashboard สร้าง HTML เปิด tab ใหม่ -> qr.js เขียน document + window.print() (หมายเหตุ: bug ที่ต้องแก้)